CitaNova

Tu salud en un clic

DOCUMENTO DE TRABAJO · VERSIÓN 1.0

Plan de mejoras y distribución del proyecto

Seguridad · Administración · Citas médicas · Experiencia de usuario

Análisis del backend Spring Boot y del frontend Angular compartidos para definir tareas concretas, responsables, dependencias, pruebas y entregables.

3 INTEGRANTES BACKEND	3 INTEGRANTES ANGULAR
Autenticación y seguridad	Login, Guards y notificaciones Toast
Usuarios, roles y permisos	Panel administrativo y permisos
Médicos, horarios y citas	Paneles de médicos y pacientes

Fecha: 14 de octubre de 2026 | **Proyecto:** CitaNova - Tu salud en un clic

Alcance: planificación basada en revisión estática de los archivos recibidos; no representa una certificación de seguridad ni una prueba de ejecución de todo el sistema.

01 · DIAGNÓSTICO DEL PROYECTO

Qué existe y qué debemos mejorar

Se revisaron los ZIP **src (1).zip** (frontend Angular) y **citas-backend(3).zip** (Spring Boot). La revisión identifica código existente y propuestas pendientes; no se ejecutó una instancia con MariaDB ni una compilación completa de Angular.

Módulo	Situación identificada	Acción prioritaria
Autenticación	Login JWT y registro público en /api/v1/auth; Guard e interceptor en Angular.	Cambio obligatorio de contraseña, control de sesión y verificación de cuentas inactivas.
Usuarios	CRUD, asignación de roles y activación/desactivación.	Búsqueda, filtros, paginación y protección del último administrador.
Roles y permisos	Enum de permisos; asignación por rol; selección individual en Angular.	Agrupar visualmente, seleccionar por módulo, contar permisos y prevenir escalada de privilegios.
Médicos	CRUD y consultas básicas; repositorio por correo/colegiado.	Búsqueda parcial por nombre/correo y filtros por especialidad/estado.
Citas	Agendar, diagnóstico, historial, agenda y cancelación.	Corregir horario cancelado, concurrencia y reprogramación.
Mensajería	Alertas locales (success/error) en componentes Angular.	Servicio y componente global de Toast coherentes.

Hallazgos técnicos prioritarios

- **JWT y cuentas desactivadas:** JwtAuthenticationFilter crea la autenticación después de validar el token sin verificar explícitamente el estado activo actual del usuario.
- **Bloqueo de cuenta:** Usuario.isAccountNonLocked() devuelve siempre true; falta política real para intentos fallidos.
- **Permisos:** RolServiceImpl.asignarPermisos() reemplaza el conjunto seleccionado sin controles adicionales sobre quién puede conceder cada permiso.
- **Citas canceladas:** cancelarCita libera el horario, pero agendarCita utiliza existsByHorario_IdHorario(), que también detecta la cita cancelada; revisar consistencia e historial.
- **Permisos en el navegador:** AuthService conserva roles/permisos en localStorage; pueden quedar desactualizados tras cambios administrativos.

02 · ORGANIZACIÓN DEL EQUIPO

Distribución de responsabilidades

Responsable	Área principal	Trabaja con
1 · Backend	Autenticación, JWT, contraseñas y auditoría de acceso.	4 · Angular
2 · Backend	Usuarios, roles, permisos y búsquedas administrativas.	5 · Angular
3 · Backend	Médicos, citas, agendas, cancelación y concurrencia.	6 · Angular
4 · Angular	Login, cambio de contraseña, Guards y Toast global.	1 · Backend
5 · Angular	Usuarios, roles, filtros, paneles de permisos y estadísticas.	2 · Backend
6 · Angular	Citas y pantallas de médicos/pacientes.	3 · Backend

Normas de colaboración

- **Un responsable por módulo:** cada integrante modifica sus archivos principales y comunica cambios de modelos, rutas y DTO a su pareja.
- **Contratos primero:** acordar ruta, método HTTP, DTO, permisos, estados y formato de error antes de construir pantallas.
- **Los Guards no sustituyen Spring Security:** cualquier autorización debe comprobarse en el servidor.
- **Ramas separadas:** usar feature/security-password, feature/admin-search, feature/appointments, feature/ui-auth-toast, feature/ui-admin, feature/ui-appointments (nombres sugeridos).
- **Integración revisada:** abrir Pull Request, ejecutar pruebas, comprobar compatibilidad y evitar subir datos privados o credenciales.
- **No duplicar componentes:** extender los servicios y pantallas actuales; centralizar Toast y errores.

Orden de coordinación

Pareja 1-4: definir bandera de cambio obligatorio y comportamiento de sesión. **Pareja 2-5:** definir contrato de búsqueda y catálogo/grupos de permisos. **Pareja 3-6:** definir estados de citas y respuesta 409 cuando un horario deje de estar libre.

INTEGRANTE 1 · SPRING BOOT

Seguridad y autenticación

Objetivo: proteger cuentas y sesiones sin reescribir el login JWT existente.

Archivos existentes que deberá revisar o modificar

- model/Usuario.java; dto/LoginResponseDTO.java; dto/LoginUsuarioDTO.java
- controller/AuthController.java; service/Impl/UsuarioServiceImpl.java
- security/JwtAuthenticationFilter.java; security/JwtService.java; config/SecurityConfig.java

Tareas asignadas

- **01.** Añadir campo persistente **debeCambiarPassword** (y migración SQL); establecer true únicamente para cuentas con contraseña temporal asignada por administración.
- **02.** Crear operación autenticada para cambiar contraseña propia; verificar la actual cuando aplique, validar la nueva y guardarla con PasswordEncoder.
- **03.** Informar en la respuesta del login si el usuario debe cambiar la contraseña; restringir en backend las operaciones disponibles hasta completarlo.
- **04.** Rechazar peticiones con JWT de usuarios desactivados; revisar vigencia de permisos, manejo 401/403 y endpoints públicos concretos.
- **05.** Implementar bloqueo temporal progresivo por fallos de login con persistencia/tiempo y mensajes que no revelen si existe el correo.
- **06.** Registrar eventos de autenticación sin contraseñas ni tokens; definir estrategia de invalidación o nueva sesión tras actualizar contraseña.
- **07.** Opcional fase 2: recuperación con token temporal de un solo uso, expiración y correo seguro.

Entregables

Migración documentada, DTO actualizados, endpoints, reglas de seguridad, pruebas JUnit y ejemplos Postman.

Criterios de aceptación

- Cuenta temporal no accede a citas, usuarios o roles antes de cambiar contraseña, incluso desde Postman.
- Cuenta desactivada no puede reutilizar un JWT previamente emitido; HTTP 401/403 correctamente distinguidos.
- Registro público de paciente no solicita cambiar la contraseña autodefinida.

Dependencias: Coordinación con integrante 4 para rutas de login/cambio de contraseña y forma de respuesta.

INTEGRANTE 2 · SPRING BOOT

Usuarios, roles y permisos

Objetivo: mejorar las operaciones administrativas y cerrar riesgos de privilegios.

Archivos existentes que deberá revisar o modificar

- controller/UsuarioController.java; repository/UsuarioRepository.java; service/Impl/UsuarioServiceImpl.java
- controller/RolController.java; service/Impl/RolServiceImpl.java; repository/RolRepository.java
- controller/PermisoController.java; model/Permiso.java; dto/RolPermisosRequestDTO.java

Tareas asignadas

- **01.** Ampliar GET de usuarios para filtrar por nombre, apellido o correo (parcial, sin distinguir mayúsculas); agregar estado/rol cuando sea necesario.
- **02.** Ampliar GET de roles con búsqueda por nombre y paginación; acordar Page/DTO sin romper consumidores Angular existentes.
- **03.** Mantener asignación actual de permisos y agregar validación del derecho a conceder cada permiso; evitar autopromoción no autorizada.
- **04.** Proteger cuentas y roles críticos: no dejar el sistema sin un administrador activo; definir quién puede modificar permisos administrativos.
- **05.** Diseñar metadatos de categorías para permisos (desde backend o contrato compartido) sin necesidad de modificar nombres persistidos del enum.
- **06.** Crear auditoría de cambios de roles, permisos y estado de usuarios; incluir actor, fecha y resultado, sin registrar secretos.
- **07.** Crear pruebas de filtros, paginación, asignaciones prohibidas y protección del último administrador.

Entregables

Búsquedas, filtros, reglas de seguridad, auditoría y contratos de respuesta documentados.

Criterios de aceptación

- Búsqueda "ana" localiza nombre/apellido/correo coincidente y respeta permisos de lectura.
- Intento de asignar permisos superiores al alcance autorizado devuelve 403 y no cambia el rol.
- No es posible desactivar o retirar privilegios al último administrador válido.

Dependencias: Coordinación con integrante 5 para búsqueda, paginación, agrupación de permisos y retorno de errores.

INTEGRANTE 3 · SPRING BOOT

Médicos, horarios y citas

Objetivo: asegurar disponibilidad correcta y permitir nuevas operaciones de agenda.

Archivos existentes que deberá revisar o modificar

- controller/MedicoController.java; repository/MedicoRepository.java; service/Impl/MedicoServiceImpl.java
- controller/ScheduleController.java; service/ScheduleService.java; repository/HorarioDisponibleRepository.java
- controller/AppointmentController.java; service/AppointmentService.java; repository/CitaRepository.java

Tareas asignadas

- **01.** Agregar búsqueda parcial de médicos por nombre, apellido o correo y filtros opcionales por especialidad/estado; preservar GET actuales.
- **02.** Corregir el conflicto entre liberar un horario al cancelar y existsByHorario_IdHorario(): definir la regla de historial que permita nuevas reservas válidas.
- **03.** Asegurar concurrencia: dos peticiones simultáneas al mismo horario no pueden generar reservas incompatibles; transacción, bloqueo/constraint y prueba paralela.
- **04.** Añadir reprogramación validando propiedad, cita elegible, nuevo horario disponible y atomicidad del cambio.
- **05.** Establecer motivo de cancelación opcional/obligatorio según regla del equipo; documentar estados de citas.
- **06.** Añadir filtros de agenda por fechas y períodos; revisar descansos y bloqueos de disponibilidad de médicos.
- **07.** Conservar y extender verificaciones actuales: historial solo del paciente autenticado y diagnóstico solo por médico asignado.

Entregables

Endpoints y DTO para consultas/reprogramación, migraciones si hacen falta y pruebas de concurrencia/autorización.

Criterios de aceptación

- Tras cancelar una cita, la disponibilidad y la posibilidad real de reservar de nuevo son coherentes.
- Dos reservas simultáneas sobre el mismo horario: una gana y otra recibe conflicto controlado (409).
- Reprogramación de cita ajena o consulta indebida devuelve 403 y no altera datos.

Dependencias: Coordinación con integrante 6 sobre filtros, calendario, estados y mensajes de reservas.

INTEGRANTE 4 · ANGULAR

Autenticación y notificaciones

Objetivo: crear una experiencia unificada de acceso, recuperación de errores y confirmaciones.

Archivos existentes que deberá revisar o modificar

- core/auth/services/auth.service.ts; core/auth/models/login-response.model.ts
- core/auth/guards/*.ts; core/auth/interceptors/auth.interceptor.ts
- features/auth/pages/login/*; app.routes.ts; app.html; app.config.ts

Tareas asignadas

- **01.** Crear página standalone y formulario reactivo de cambio de contraseña; validaciones y mensajes de seguridad.
- **02.** Actualizar modelo LoginResponse y navegación: si debeCambiarPassword es true, redirigir al cambio obligatorio.
- **03.** Adecuar Guards/interceptor y refresco de permisos para evitar navegación improcedente; backend sigue siendo autoridad final.
- **04.** Crear shared/notifications o core/notifications con **NotificationService** y componente **ToastContainer** montado una sola vez.
- **05.** Soportar éxito, error, advertencia e información; autocierre, botón cerrar, accesibilidad y evitar mensajes repetidos.
- **06.** Centralizar tratamiento de 401, 403, 409 y 500; no revelar stack traces ni información sensible.
- **07.** Actualizar confirmaciones críticas: usar un diálogo reutilizable sin depender solamente de window.confirm().

Entregables

Pantalla cambio de contraseña, ajustes auth, Toast global, diálogo de confirmación, guía breve de uso y pruebas.

Criterios de aceptación

- Al guardar correctamente aparece Toast de éxito; ante respuesta 403 o 409 aparece mensaje útil y seguro.
- Un único servicio permite invocar success/error/warning/info desde los módulos 5 y 6.
- Navegación obligatoria acorde al estado de contraseña y permisos reales.

Dependencias: Esperar contrato de backend 1; publicar API del NotificationService temprano para desbloquear integrantes 5 y 6.

INTEGRANTE 5 · ANGULAR

Panel administrativo

Objetivo: facilitar la gestión de usuarios, médicos, roles y permisos de CitaNova.

Archivos existentes que deberá revisar o modificar

- features/admin/users/pages/user-list/*; features/admin/users/services/user.service.ts
- features/admin/roles/role-list/*; features/admin/roles/role-form/*; features/admin/roles/services/role.service.ts
- features/admin/doctors/doctor-list/*; features/admin/admin-dashboard/*

Tareas asignadas

- **01.** Agregar buscador con filtros por nombre/correo y estado a usuarios y médicos; conectar parámetros de backend y evitar peticiones por cada tecla (debounce).
- **02.** Incorporar búsqueda y paginación a listado de roles, con estados vacío, cargando y error.
- **03.** Cambiar etiquetas individuales de permisos en tabla por "N permisos asignados" y acción "Ver permisos".
- **04.** Crear modal para ver permisos agrupados: Administración, Médicos, Pacientes, Citas, Horarios y Especialidades.
- **05.** Extender selectedPermissions y togglePermission() existentes con seleccionar todos, seleccionar categoría y limpiar, mostrando contador.
- **06.** Usar los Guards y permisos existentes para ocultar acciones, pero mantener validaciones de backend y manejo de 403.
- **07.** Reemplazar alertas locales por NotificationService y mensajes de confirmación antes de cambios importantes.
- **08.** Dashboard administrativo: estadísticas reales solo cuando exista endpoint acordado; evitar números simulados en producción.

Entregables

Listados filtrables, modal de permisos, selección masiva, dashboard mejorado y pruebas de componentes.

Criterios de aceptación

- La tabla no enumera 30 permisos en una celda; detalles visibles a demanda.
- Seleccionar por categoría no borra permisos de otras categorías; al guardar se persiste la selección exacta.
- Filtros funcionan con parámetros reales y responden correctamente a errores de acceso.

Dependencias: Backend 2 define filtros y permisos; Angular 4 publica NotificationService y confirmaciones.

INTEGRANTE 6 · ANGULAR

Médicos y pacientes

Objetivo: simplificar agenda, reservas, historial y consulta de especialistas.

Archivos existentes que deberá revisar o modificar

- features/patient/patient-doctors/*; features/patient/patient-schedules/*
- features/patient/patient-new-appointment/*; features/patient/patient-history/*
- features/doctor/doctor-appointments/*; features/doctor/doctor-schedules/*
- features/patient/appointment.service.ts; features/doctor/doctor.service.ts

Tareas asignadas

- **01.** Agregar búsqueda por médico (nombre/correo) y filtros por especialidad/estado; conectarlos al backend existente ampliado.
- **02.** Desarrollar vista calendario o agenda por día/semana sin duplicar datos del servidor.
- **03.** Diseñar formulario de reprogramación y confirmar disponibilidad justo al guardar, mostrando conflictos 409 cuando corresponda.
- **04.** Mejorar historial y estados de citas con etiquetas comprensibles; añadir motivo de cancelación si backend lo exige.
- **05.** Facilitar panel del médico: citas del día, selección por fecha y acceso solo a su propia agenda.
- **06.** Usar Toast en creación/cancelación/reprogramación y confirmaciones para acciones irreversibles.
- **07.** Probar flujos vacíos, fechas no disponibles, citas ajenas, errores HTTP y pantallas responsive.

Entregables

Buscador de médicos, vistas de agenda, reprogramación, notificaciones y pruebas de interacción.

Criterios de aceptación

- Una búsqueda por especialidad filtra la lista sin mostrar información no autorizada.
- Si otro paciente reserva antes el horario, se informa el conflicto y se actualizan los horarios.
- El médico no puede acceder a información de otra agenda mediante parámetros manipulados.

Dependencias: Backend 3 publica reglas de horarios/citas; Angular 4 publica NotificationService.

09 · CONTRATOS DE API Y TOAST

Acuerdos que deben quedar cerrados

Estas rutas se proponen para coordinar el desarrollo. **No se consideran implementadas** en los ZIP, salvo las identificadas como existentes. Ajustar nombres finales a los controladores reales.

Método / ruta	Estado	Propósito
POST /api/v1/auth/login	Existente	Autenticación; ampliar respuesta con indicador de cambio obligatorio.
POST /api/v1/auth/register	Existente	Registro público con contraseña propia.
PUT /api/v1/me/password	Propuesta	Cambiar contraseña propia; operación protegida.
GET /api/v1/me	Propuesta	Consultar estado y permisos vigentes de la sesión.
GET /api/v1/users?search=...	Ampliación	Buscar nombre, apellido o correo; filtros/paginación.
GET /api/v1/roles?search=...	Ampliación	Filtrar roles por nombre y paginar.
GET /api/v1/medicos?search=...	Ampliación	Buscar médicos y filtrar por especialidad.
PUT /api/v1/appointments/{id}/reschedule	Propuesta	Cambiar horario de una cita válida y propia.
GET /api/v1/appointments/my-agenda	Existente	Agenda propia del médico por fecha; ampliar si se necesita.

Contrato mínimo del sistema Toast

- **API Angular:** notification.success(message), notification.error(message), notification.warning(message) y notification.info(message).
- **Diseño:** aviso superior derecho o inferior derecho; cierre automático 4-6 s, cierre manual, máximo de avisos visibles y soporte para teclado.
- **Casos de uso:** crear/editar/eliminar; asignar permisos; login; cambios de contraseña; citas y conflictos de horario.
- **Seguridad:** no mostrar contraseñas, JWT, detalles internos, trazas del servidor o campos privados.
- **Coordinación:** el interceptor no debe duplicar los mensajes del componente ante el mismo error.

Ejemplo de errores acordados

400: datos incorrectos · **401:** autenticación inválida o sesión expirada · **403:** operación prohibida · **404:** recurso inexistente · **409:** conflicto de reserva o cambio · **500:** fallo imprevisto con mensaje genérico.

10 · CRONOGRAMA Y ENTREGAS

Propuesta de ejecución en dos semanas

Periodo	Backend (1-3)	Angular (4-6)	Entrega para integrar
Días 1-2	Revisar contratos, migraciones y pruebas existentes; cerrar riesgos críticos.	Preparar modelos, rutas y Toast básico reutilizable.	Contrato de API aprobado.
Días 3-5	Cambio obligatorio de contraseña; búsquedas; corrección citas canceladas.	Pantalla de contraseña; filtros y modal de permisos; buscador médicos.	Primeras funciones probables con Postman.
Días 6-8	Bloqueo de cuenta, reglas de permisos, concurrencia y reprogramación.	Selección masiva, calendario y Toast integrado.	Pull Requests funcionales por módulo.
Días 9-10	Auditoría, pruebas de autorización y conflictos.	Integración con endpoints reales y manejo de estados.	Pruebas cruzadas entre parejas.
Días 11-12	Corrección de errores y regresión.	Responsive, usabilidad, tests y accesibilidad.	Versión candidata.
Días 13-14	Pruebas de seguridad y revisión de base de datos.	Build final, pruebas end-to-end y demo.	Entrega integrada y documentación.

Lista de control para cada Pull Request

- Contiene únicamente cambios del módulo asignado y enlaza la tarea acordada.
- No modifica nombres de endpoint ni DTO sin informar a su pareja y al equipo.
- Incluye pruebas y evidencia de resultado (Postman o captura de interfaz).
- Si hay migraciones SQL, se incluyen scripts reversibles o instrucciones de respaldo.
- No incluye application.properties con credenciales, JWT secretos, ZIP, logs ni node_modules.
- La integración a main se hace después de revisión y no mediante push forzado.

Nota: el cronograma es orientativo. Si el equipo dispone de menos tiempo, priorizar contraseña obligatoria, cuentas desactivadas, autorizaciones, disponibilidad de citas, permisos agrupados y Toast; dejar 2FA, recuperación por correo y auditoría avanzada como segunda fase.

11 · PRUEBAS Y FINALIZACIÓN

Criterios de aceptación global

Cada funcionalidad debe verificarse en el servidor y en Angular. Los siguientes casos constituyen el mínimo de pruebas cruzadas del equipo.

Prueba	Resultado esperado	Equipo
Contraseña temporal	No se permite usar endpoints de negocio hasta completar el cambio obligatorio.	1 + 4
JWT de usuario inactivo	Token emitido antes de desactivar deja de autorizar nuevas peticiones.	1 + 4
Autopromoción de permisos	Un usuario sin autorización no puede asignar privilegios elevados.	2 + 5
Último administrador	No es posible dejar el sistema sin administrador operativo.	2 + 5
Búsquedas parciales	Coinciden nombre/apellido/correo sin distinción de mayúsculas.	2,3 + 5,6
Selección masiva	Seleccionar/limpiar una categoría no altera otras; total exacto.	5
Reserva concurrente	Solo una reserva válida; otra recibe conflicto coherente.	3 + 6
Reserva tras cancelación	Historia conservada y disponibilidad consistente.	3 + 6
Acceso a citas de terceros	Petición ajena o modificación prohibida devuelve 403.	3 + 6
Toast uniforme	Éxito/error/aviso se muestran sin duplicados ni información sensible.	4,5,6

Observaciones de seguridad y alcance

- Se identificó una regla pública amplia **/api/v1/auth/**** en SecurityConfig; al agregar endpoints sensibles, reemplazarla por rutas públicas específicas.
- Las contraseñas actuales usan BCrypt; conservar un hash seguro, ampliar longitud aceptada y no registrar ni devolver credenciales.
- El frontend conserva token y datos de usuario en localStorage; evaluar riesgos XSS y una estrategia de sesión más robusta según arquitectura y tiempo.
- La carpeta Angular compartida incluye src, pero no todo el entorno de compilación. Ejecutar build y pruebas en el repositorio completo.
- Backend: inspección de código fuente; revisar pruebas con Maven y una MariaDB de pruebas antes de dar por cerradas las tareas.

Decisión de salida: considerar la mejora lista solamente cuando la pareja backend/frontend haya demostrado la operación funcional, el caso fallido correspondiente y la regresión de las funcionalidades existentes.

Referencias para el equipo: ZIP Angular «src (1).zip» y ZIP Spring Boot «citas-backend(3).zip», revisados estáticamente el 14 de octubre de 2026. Las rutas de archivos indicadas en cada asignación son relativas a src/app (Angular) y src/main/java/com/proyectocitas (backend), respectivamente.